DepChain

André Pires (ist1116452)[1], Pedro Duarte (ist1116390)[1], and Tomás Fernandes (ist1116122)[1]

Instituto Superior Técnico,
Universidade de Lisboa,
Av. Rovisco Pais 1, 1049-001 Lisboa, Portugal
{andre.vaz.pires, pedro.alegre.duarte, tomas.santos.f}@tecnico.ulisboa.pt
https://tecnico.ulisboa.pt

Abstract. This report presents DepChain, a permissioned blockchain built over the Basic HotStuff Byzantine Fault Tolerant consensus protocol. Starting from a dependable stage-1 substrate based on Authenticated Perfect Links over UDP and threshold-signed Quorum Certificates (QCs), the system is extended with an account-based blockchain state, fee-ordered multi-transaction blocks, native (DepCoin) and token (IST Coin) operations, gas accounting and smart-contract execution through the Hyperledger Besu EVM. The design enforces dependability mechanisms, such as authentication and account-level authorization, non-repudiation, replay resistance, double-spend prevention, deterministic execution across replicas, and resistance to Byzantine clients and replicas, including collusion between them. A comprehensive test suite, demonstrates robustness under this Byzantine system behavior.

Keywords: Byzantine Fault Tolerance (BFT) · Basic HotStuff · Permissioned Blockchain · Smart Contracts · ERC-20 · Gas Accounting · Dependability.

1 Introduction

DepChain is a permissioned blockchain replicated by $n = 3f+1$ statically configured nodes under a Byzantine fault model. Its goal is not only to reach consensus on blocks, but also to preserve application-level dependability guarantees once transactions and smart contracts are introduced. In particular, the final system must ensure guarantees such as authentication and account-level authorization, non-repudiation, replay resistance, double-spend prevention, non-negative balances, deterministic execution across honest replicas and robustness against Byzantine clients and leaders/replicas.

The project was developed in two stages. Stage 1 established the dependable substrate required to run consensus over raw UDP, progressively transforming unreliable datagram delivery into authenticated communication and implementing Basic HotStuff with threshold-signed QCs. Stage 2 then built the blockchain semantics on top of this substrate: an account-based state, with multi-transaction blocks, native DepCoin operations, and EVM-based smart-contract execution.

The remainder of the report first summarizes the dependable communication and consensus substrate, then presents the blockchain and smart-contract extensions introduced in stage 2, and finally discusses the main threats, countermeasures and test evidence supporting the system guarantees.

2 Design

2.1 Communication Substrate

DepChain is built over a layered communication stack that transforms raw UDP into authenticated and reliable point-to-point channels. The *UDP Fair-Loss Link* handles packet transmission and manually configured fault injection (loss, duplication, corruption and delay), allowing controlled adversarial testing. On top of it, the *Stubborn Link* retransmits messages periodically until they are acknowledged. The *Perfect Link* adds per-sender sequence numbers to provide exactly-once, in-order delivery by filtering duplicastes and buffering out-of-order messages. Finally, the *Authenticated Perfect Link* (APL) authenticates each delivered payload with an HMAC-SHA256 tag bound to both the payload and sequence number, preventing tampering and replay at the link layer. Session keys are derived pairwise through ECDH during process initialization, which is acceptable under the project's static-membership assumption. This substrate is shared by both replicas and clients.

2.2 Cryptographic Mechanisms

DepChain uses two different authentication mechanisms for two different purposes. For point-to-point channel authentication, we use HMAC-SHA256 because it is substantially cheaper than public-key signatures and is sufficient to guarantee pairwise authenticity and integrity over authenticated links. A trade-off would be with key proliferation, which scales quadratically, but, again, this is acceptable under our static-membership model. For quorum certificates (QCs), however, pairwise MACs are insufficient: the QC must be publicly verifiable and aggregatable across replicas. For this reason, we use BLS12-381 threshold signatures through the `herumi/bls` library [1] (*"an implementation of BLS threshold signature, which supports the new BLS Signatures specified at Ethereum 2.0 Phase 0"* [1]). Each replica holds a secret share of the signing key and independently produces a partial signature over its HotStuff vote. The leader combines any $2f + 1$ valid partial signatures into a single threshold signature, yielding a compact QC without requiring an additional interactive signing round. This choice keeps QC formation compatible with the HotStuff vote process and avoids the fragility of multi-round threshold-signing protocols in the presence of Byzantine or crashed replicas.

2.3 Basic HotStuff Core

Consensus follows the four Basic HotStuff [2] phases: *PREPARE*, *PRE-COMMIT*, *COMMIT* and *DECIDE*. In each view, the leader collects $2f + 1$ *NEW_VIEW*

messages, selects the highest QC among them, extends the corresponding branch with a new proposal and starts the voting pipeline. Replicas vote only when the proposal satisfies the `safeNode` predicate, i.e., itt either extends the currently locked branch or carries a justification from a strictly higher view. This preserves HotStuff safety by preventing honest replicas from voting for conflicting unsafe branches. [2]

The implementation includes several mechanisms required for robust execution in an asynchronous environment. First, replicas buffer *NEW_VIEW* messages that arrive for future views and later consume them immediately when they become leader of that view, instead of waiting for retransmission. Second, view-change timers use exponential backoff, reducing perpetual desynchronization when replicas start at slightly different times. Third, the proposal thread sleeps until both a quorum of *NEW_VIEW* messages and a non-empty mempool are available, avoiding busy-waiting. Finally, duplicate *DECIDE* messages do not trigger repeated execution: each decided block is applied at most once, and after commitment the implementation prunes sibling branches from the HotStuff tree, preserving only the committed branch and removing stale non-decided alternatives.

2.4 Blockchain State and Blocks

Stage 2 extends the consensus core with an account-based blockchain state. The world state contains externally owned accounts (EOAs), contract accounts, native DepCoin balances, nonces, contract code and selected tracked storage slots. The system starts from a genesis block that initializes the native balances and deploys the IST Coin contract at a fixed address. Subsequent committed blocks persist the ordered transaction list, execution receipts, proposer identity and resulting deterministic state hash. The state hash is computed as the SHA-256 digest of the concatenated canonical serialization of every account entry, including address, nonce, native balance, contract code and tracked storage slots, ensuring that any divergence in any field is detected and that all honest replicas can verify state convergence after each block.

Blocks contain multiple transactions. Proposal ordering is fee-driven, but must remain deterministic and compatible with account nonces. We therefore order transactions by descending maximum fee, while strictly preserving per-sender nonce order. Equal-fee ties are broken deterministically using the transaction hash. This guarantees that all honest replicas build and interpret blocks identically.

2.5 Transaction Model, Nonces and Client Requests

All client operations are represented by signed requests carrying signed transactions. This includes native transfers, smart-contract calls and committed-state balance queries. The outer client request is signed by the client identity and

authenticated by replicas before admission into the mempool, preventing spoofing or impersonation. The inner transaction is signed separately, and its signer-derived address must match the transaction sender, which enforces account-level authorization and non-repudiation.

Replay protection operates at two levels. At the request layer, replicas track requests by $(clientId, requestId)$ and ignore duplicates that are already pending or already executed. At the blockchain layer, transactions carry account nonces. The implementation accepts future nonces to support pipelining and out of order arrival, but rejects stale already committed nonces and duplicate pending nonces. This is a deliberate tradeoff: strict next nonce admission would simplify the mempool, but would also be too rigid in an asynchronous BFT setting where valid operations may arrive before the replica catches up with the state that makes them executable (e.g., delayed requests).

2.6 Smart Contracts and Gas Accounting

Smart-contract execution is implemented with the Hyperledger Besu EVM. Native DepCoin operations use a fixed defined gas cost, while contract calls and deployments are charged using the actual gas consumed by the EVM execution. The effective fee follows the project rule $\min(gasPrice \times gasLimit,\ gasPrice \times gasUsed)$ so unused gas is refunded to the sender, whereas out-of-gas executions charge the full gas limit and do not change the state. Execution produces a receipt containing the transaction hash, success flag, gas used, charged fee, return data and deployed contract address when applicable.

To satisfy the stage 2 requirements, the ERC-20 IST Coin contract was adapted to resist the classic approval frontrunning attack. Unsafe non-zero to non-zero allowance replacement through `approve` is rejected, while allowance changes are performed through `increaseAllowance` and `decreaseAllowance`. This prevents a spender from exploiting an allowance-reduction race to benefit from both the old and the new allowance.

2.7 Client-Side Confirmation

Clients broadcast requests to all replicas and collect authenticated replies. A request is considered finalized only when the client receives $f + 1$ matching committed replies from distinct replicas. Reply matching is based on a canonical response identity derived from all response fields, preventing a Byzantine replica from counting multiple times or equivocating about the committed result. For read operations, this mechanism provides quorum confirmed committed state snapshots, and for write operations, it provides end-to-end confirmation that the transaction was ordered, executed and committed consistently by a quorum of replicas. Another important point is that clients submit requests asynchronously. After broadcasting a request, the client waits only for $f+1$ matching `ACCEPTED` or `REJECTED` replies, which indicate whether the request entered processing or was discarded. Finalization, however, is treated separately from admission: read and write operations are still ordered through consensus so that the observed result

is consistent with committed system state rather than with a potentially stale snapshot taken while other transactions are still pending or executing. This is a deliberate design tradeoff between latency and freshness. Gas pricing therefore also acts as a prioritization mechanism: clients willing to pay a higher fee are more likely to have their requests included and resolved earlier.

3 Dependability Analysis

This section presents each dependability guarantee together with the threats it must withstand and the concrete countermeasures the design provides.

3.1 Consensus Safety

Guarantee. No two honest replicas commit conflicting blocks for the same height.

Threats. A Byzantine leader may send conflicting *PREPARE* proposals, inject forged or unsigned transactions, tamper with client metadata or propose blocks that violate the HotStuff locking rules. Byzantine replicas may send invalid partial signatures or a Byzantine leader may inject a forged QC to move honest replicas onto inconsistent branches.

Countermeasures. Safety follows from HotStuff's three phase locking: a replica votes *COMMIT* only after seeing a valid `preCommitQC` that aggregates $2f + 1$ partial *PRE-COMMIT* signatures. The `lockedQC` mechanism ensures replicas reject proposals that do not extend the highest locked block. Honest replicas revalidate every proposed block before voting, by checking QC validity, enforcing the `safeNode` predicate, verifying each transaction signature and signer derived sender address, and rejecting blocks whose transaction list and request metadata are inconsistent. Partial BLS signatures are combined into a QC only when enough valid votes are available, and the resulting threshold signature is verified against the shared public key. A single Byzantine actor therefore cannot construct a valid QC, and malformed or equivocated proposals do not collect enough honest votes to form one. After a block is decided and executed, the implementation prunes sibling branches from the local HotStuff tree, ensuring that stale non-decided alternatives do not remain indefinitely in the replica state.

3.2 Consensus Liveness

Guarantee. Under partial synchrony, progress eventually resumes once messages are delivered and an honest leader drives a view.

Threats. A Byzantine leader may refuse to propose or otherwise stall its view, preventing progress indefinitely.

Countermeasures. Replicas employ timeout-based view changes with exponential backoff: once the timeout expires, they advance to the next view and send *NEW_VIEW* to the next leader. Buffered future view messages are retained and consumed immediately when a replica later becomes leader of that

view, improving recovery under asynchronous delivery. Together, these mechanisms prevent the system from remaining stuck behind a silent or faulty leader.

3.3 Authenticated and Reliable Communication

Guarantee. Between correct processes, the communication stack provides integrity, authenticity, "exactly once delivery" and "in order delivery" despite UDP loss, duplication, delay or corruption.

Threats. Because DepChain runs directly over UDP, the network substrate must tolerate message loss, duplication, delay, reordering, tampering and replay.

Countermeasures. Loss is handled by the Stubborn Link retransmission mechanism. The Perfect Link uses per-sender sequence numbers to filter duplicates and deliver messages in order. The Authenticated Perfect Link (APL) attaches an HMAC-SHA256 tag bound to both the message and its sequence number, so any tampering or replay attempt is detected and discarded before reaching the protocol logic. Since session keys are derived pairwise through ECDH, an attacker without the corresponding secret material cannot impersonate another process at the link layer.

3.4 Authorization and Non-Repudiation

Guarantee. Only the owner of an account can authorize a transaction from it, and the signer cannot later deny having issued the transaction.

Threats. A Byzantine client or network adversary may forge or replay client requests, spoof client identities or inject unauthorized transactions into the mempool. A Byzantine leader colluding with a Byzantine client may include transactions from unknown senders or tamper with transaction fields.

Countermeasures. Each client request carries an outer client signature verified against the trusted public key of the claimed client identity, preventing spoofed clientIds or forged envelopes from entering the mempool. The enclosed blockchain transaction carries its own signature, and replicas require the signer derived blockchain address to match the transaction sender. Even if a Byzantine leader bypasses mempool admission and injects a malformed proposal directly into the HotStuff pipeline, honest replicas revalidate the proposed block before voting, including transaction signatures, sender binding and request metadata consistency. This two layer verification provides both account-level authorization and non-repudiation.

3.5 Replay Resistance

Guarantee. Replay of old requests does not lead to repeated state changes, and repeated *DECIDE* deliveries do not cause duplicate block execution.

Threats. A Byzantine client or network adversary may replay previously seen requests, while delayed or duplicated *DECIDE* messages may attempt to trigger repeated execution of the same block.

Countermeasures. At the request layer, replicas track requests by $(clientId, requestId)$ and reject duplicates already pending or already executed. At the blockchain layer, stale nonces already committed are rejected and duplicate pending nonces are discarded. A replayed signed transaction under a fresh request identifier still cannot re-execute once its original nonce has been consumed. Finally, block execution is guarded by an idempotence check so the same decided block is applied at most once.

3.6 Financial Safety

Guarantee. Native DepCoin balances remain non-negative, transfers with conflicting nonces cannot both commit, overspending attempts fail deterministically, and no value is created or destroyed except for the intended transfer of gas fees to the block proposer.

Threats. A Byzantine client may submit conflicting transfers with the same nonce, replay stale transactions or attempt to exploit future nonces to double-spend.

Countermeasures. Stale nonces already committed are rejected, duplicate pending nonces are rejected and block construction preserves per-sender nonce order. In addition, replicas validate proposed blocks on a snapshot state before voting, advancing sender nonces across accepted transactions so that a Byzantine leader cannot include nonce conflicts from same sender into a single block. The implementation deliberately accepts future nonces to support pipelining and out-of-order arrival, but this is controlled by duplicate-pending checks and deterministic execution order.

3.7 Deterministic Execution and Replica Convergence

Guarantee. For the same committed block, all honest replicas derive the same transaction ordering, receipts, block hash and resulting state hash, including smart-contract executions that revert.

Threats. Non-deterministic transaction ordering or execution could cause replicas to diverge silently, undermining the state-machine replication guarantee.

Countermeasures. Transactions are ordered by descending maximum fee with per-sender nonce order strictly preserved, while equal-fee ties are broken deterministically by transaction hash. Smart-contract execution uses the deterministic Besu EVM, and the resulting state hash is computed deterministically over the tracked account state and tracked storage values relevant to the supported contracts. Both successful and reverting contract outcomes therefore converge identically across replicas.

3.8 Query (Read Operations) Consistency

Guarantee. Balance queries return up-to-date values of the committed state rather than speculative intermediate values.

Threats. Read operations could expose speculative state from in-flight consensus rounds, causing clients to act on values that may never be committed.

Countermeasures. DepChain ensures that queries reflect the last committed state even when other transactions are concurrently being proposed or executed. Read requests are included in the consensus process so that clients receive quorum-confirmed committed state snapshots. As discussed earlier, the tradeoff is added latency for reads, but this prevents exposure of uncommitted values.

3.9 ERC-20 Approval Frontrunning Resistance

Guarantee. A spender cannot exploit an allowance reduction race to benefit from both the old and the new allowance.

Threats. In the classic frontrunning attack, a token owner calls `approve` to reduce a spender's allowance. The spender observes the pending transaction and races to spend the old allowance before the reduction takes effect, then spends the new allowance as well, consuming both.

Countermeasures. The IST Coin contract rejects unsafe non-zero to non-zero `approve` replacements. The `approve` entry point is retained only for safe transitions such as $0 \to X$ and $X \to 0$, while allowance adjustments are otherwise performed through `increaseAllowance` and `decreaseAllowance`. This removes the classic overwrite-based ERC-20 race.

4 Testing

The test suite was designed around dependability guarantees, rather than isolated functionality. We therefore split validation into two complementary layers. First, unit tests check deterministic local behavior of the state machine, transaction validation, gas accounting, serialization, ordering and edge-case robustness. Second, integration tests execute the full client $\to$ mempool $\to$ HotStuff $\to$ execution $\to$ persistence pipeline across four replicas and explicitly inject Byzantine behaviors such as forged requests, malformed leader proposals, replayed operations, collusion attempts, repeated *DECIDE* delivery and faulty-view accumulation. This structure lets us argue both that the local transition function is correct and that the distributed protocol preserves that correctness under adversarial conditions. Table 1 in Appendix A summarizes the mapping of guarantees to test classes.

Deterministic execution and local correctness. At unit level, `BlockBuilderOrderingTest` and `BlockBuildingAndPersistenceTest` show that block construction is deterministic: transactions are ordered by fee while preserving per-sender nonce order, equal-fee ties are broken deterministically, and identical inputs produce identical block contents and hashes. `TransactionExecutionTest`, `GasParameterEdgeCasesTest` and `TransactionExecutorErc20Test` validate native transfer semantics, gas charging and refund rules, out-of-gas behavior, and ERC-20 execution against the Besu EVM. `BlockSerializerReceiptRoundTripTest`,

`ClientResponseHelperTest` and `ClientResponseReceiptFieldsTest` verify that receipts, gas fields, return data and committed responses are preserved across serialization and client-side decoding. Finally, `EdgeCaseRobustnessTest` exercises non-common but valid cases, such as self-transfers, transfers to the zero address and safe ERC-20 allowance corner cases.

Authorization, replay resistance and financial safety. A second group of tests targets the guarantees that only authorized users may change state and that operations are not applied multiple times. `TransactionValidationTest` rejects missing or forged signatures and signer/sender mismatches, while `InvalidOuterSignatureTest` shows that forged, missing or tampered outer client-request signatures are dropped before the request reaches consensus. Replay protection is covered both locally and end-to-end: `ConsensusIntegrationTest` checks mempool deduplication, `RequestReplayThroughConsensusTest` replays the same request and the same signed transaction through the full protocol, and `RepeatedDecideIdempotenceTest` confirms that repeated *DECIDE* delivery does not trigger duplicate execution. Financial safety is exercised by `DoubleSpendConsensusTest`, `InsufficientBalanceAfterTransferTest` and `FutureNonceAndPipeliningTest`, which show that conflicting transfers cannot both commit, overspending attempts fail deterministically, stale committed nonces are rejected, and future nonces are handled according to the chosen pipelining policy.

Byzantine leaders, collusion and malformed consensus inputs. To validate resistance against Byzantine protocol participants, we implemented adversarial integration tests that go beyond the stage 1 silent-leader and equivocation scenarios. `ByzantineLeaderMalformedBlockTest` checks that honest replicas reject proposals carrying forged or unsigned transactions. `ByzantineClientLeaderCollusionTest` strengthens this by modeling collusion between a Byzantine client and a Byzantine leader, covering unknown senders, tampered transactions and mismatched request metadata. `ByzantineDivergentQcInjectionTest` verifies that forged divergent QCs do not move honest replicas onto inconsistent branches. `ConsensusRaceProtectionTest` further checks that blocked or delayed execution does not violate safety and that in-flight protocol activity does not cause duplicate commits. Finally, `PruningStressTest` stresses the HotStuff tree under repeated faulty views and confirms that stale branches are pruned after commits. Together, these tests support the claim that malformed proposals, forged certificates, collusion attempts and branch accumulation are all handled safely by honest replicas.

Smart contracts, reads and replica convergence. Stage 2 also required contract- and read-specific testing. `NativeTransferHotStuffTest` and `Erc20HotStuffTest` validate the full native and ERC-20 operation lifecycle through consensus, including transfers, allowance updates, committed-state queries and rejection of unsafe approval replacement. `ApprovalFrontrunningResistanceTest` specifically

demonstrates that a spender cannot exploit an allowance-reduction race to benefit from both the old and the new allowance. `Erc20RevertReceiptConsistencyTest` shows that both successful and reverting smart-contract calls produce identical receipts, block hashes and state hashes across replicas, while `EqualFeeOrderingConsensusTest` confirms deterministic ordering and identical receipts even in equal-fee tie cases. Since reads are modeled as consensus transactions in the final design, `ReadTest`, `ReadTimeoutTest` and `QueryConsistencyTest` validate correct read semantics both when consensus succeeds and when quorum is unavailable. Finally, `StressTest` checks a global invariant under concurrent transfers: no native currency is created or destroyed except for the intended transfer of gas fees to proposers.

5 Conclusion

DepChain combines a dependable stage 1 substrate, the authenticated communication over UDP and Basic HotStuff with threshold signed QCs, with a stage 2 blockchain state machine supporting native transfers, fee-ordered blocks, gas accounting and Besu-based smart-contract execution. The final design preserves the main project guarantees: consensus safety and recovery under view changes, authorization and non-repudiation of transactions, replay and double-spend resistance, deterministic execution and replica convergence, and resistance to Byzantine clients, Byzantine leaders and collusion between them. These guarantees are supported by a comprehensive test suite, including adversarial end-to-end scenarios.

References

1. Herumi: BLS threshold signature library. `https://github.com/herumi/bls`, accessed: 09 March 2026
2. Yin, M., Malkhi, D., Reiter, M.K., Golan Gueta, G., Abraham, I.: HotStuff: BFT consensus in the lens of blockchain. CoRR **abs/1803.05069** (2019), `https://arxiv.org/abs/1803.05069`

A Test Mapping

Table 1. Mapping of dependability guarantees to test classes.

Guarantee	**Test Classes**
Consensus safety	`ByzantineLeaderMalformedBlockTest`, `ByzantineDivergentQcInjectionTest`, `ConsensusRaceProtectionTest`, `PruningStressTest`
Authorization & non-repudiation	`TransactionValidationTest`, `InvalidOuterSignatureTest`, `ByzantineClientLeaderCollusionTest`
Replay resistance & idempotence	`ConsensusIntegrationTest`, `RequestReplayThroughConsensusTest`, `RepeatedDecideIdempotenceTest`
Financial safety	`DoubleSpendConsensusTest`, `InsufficientBalanceAfterTransferTest`, `FutureNonceAndPipeliningTest`, `StressTest`
Deterministic execution & convergence	`BlockBuilderOrderingTest`, `BlockBuildingAndPersistenceTest`, `EqualFeeOrderingConsensusTest`, `Erc20RevertReceiptConsistencyTest`
Read correctness & query consistency	`ReadTest`, `ReadTimeoutTest`, `QueryConsistencyTest`
ERC-20 frontrunning resistance	`ApprovalFrontrunningResistanceTest`, `Erc20HotStuffTest`
Serialization & response correctness	`BlockSerializerReceiptRoundTripTest`, `ClientResponseHelperTest`, `ClientResponseReceiptFieldsTest`
Gas accounting	`TransactionExecutionTest`, `GasParameterEdgeCasesTest`, `TransactionExecutorErc20Test`
Edge-case robustness	`EdgeCaseRobustnessTest`